

CAPSTONE PROJECT REPORT

SharkTrack AI: Real-Time Shark Foraging Habitat Monitor & Telemetry System

Submitted by

Project Group Number : [Insert Group Number]

Course Code : [Insert Course Code]

Under the Guidance of Dr. Anshu Vashisth

School of Computer Science and Engineering

[Insert University Logo Here]

Registration Number,Name

[Reg No.],Abhishek Kumar Yadav

[Reg No.],Abdul Kadir

[Reg No.],Rajdeep Ghosh

[Reg No.],Mansi Kumari

[Reg No.],Sania

TABLE OF CONTENTS

Chapter 1 – Introduction	3
1.1 Background and Ecological Context	3
1.2 Problem Statement	4
1.3 Objective of the Project	5
1.4 Project Scope and Limitations	6
1.5 Use Case Scenarios	7
Chapter 2 – System Analysis and Requirement Specification	9
2.1 Stakeholder Analysis	9
2.2 Existing Systems vs. Proposed Solution	10
2.3 Feasibility Study	11
2.4 Functional Requirements	12
2.5 Non-Functional Requirements	13
2.6 System Dependencies and Assumptions	14
Chapter 3 – System Design and Architecture	16
3.1 High-Level System Architecture	16
3.2 Module Decomposition	17

3.3 Database Design and Schema	19
3.4 Data Flow Diagrams (DFD)	21
3.5 User Interface Design Philosophy	23
Chapter 4 – Machine Learning Implementation	25
4.1 Data Acquisition and Preprocessing	25
4.2 Feature Engineering and Selection	26
4.3 Model Selection and Hyperparameter Tuning	27
4.4 Model Validation and Performance Metrics	29
4.5 Inference Pipeline and Optimization	30
Chapter 5 – Implementation and Technologies	32
5.1 Technology Stack Justification	32
5.2 Backend Implementation (FastAPI)	33
5.3 Frontend Implementation (React)	34
5.4 IoT Simulation Layer	35
Chapter 6 – Project Scheduling and Resource Management	37
6.1 Project Lifecycle and Gantt Chart	37
6.2 Resource Allocation	38
Chapter 7 – Conclusion and Future Scope	39
7.1 Conclusion	39
7.2 Future Scope	40
References	41

Chapter 1 – Introduction

1.1 Background and Ecological Context

The world’s oceans are facing an unprecedented crisis. Apex predators, particularly sharks, are experiencing catastrophic population declines, with recent international assessments indicating that over one-third of all shark and ray species are threatened with extinction. These animals are not merely charismatic megafauna; they are keystones of marine ecosystems. By regulating prey populations and removing sick or weak individuals, sharks maintain the resilience and balance of complex food webs. The loss of these predators can trigger trophic cascades, leading to the

collapse of commercial fisheries and the degradation of coral reef systems.

However, protecting these animals is a formidable challenge. Unlike terrestrial animals that can be fenced into national parks, pelagic sharks inhabit a vast, three-dimensional, and fluid environment. They are highly migratory, traversing thousands of kilometers across international borders to follow dynamic oceanographic features such as thermal fronts, upwelling zones, and mesoscale eddies. These features, often rich in nutrients and prey, are ephemeral—they move, grow, and dissipate over days or weeks.

1.2 Problem Statement

Traditional marine conservation strategies rely heavily on **Marine Protected Areas (MPAs)**, which are static geographic boundaries drawn on a map. While effective for sedentary species, static MPAs often fail to protect highly mobile sharks. A region designated as a "protected zone" today may be ecologically barren next week if the nutrient-rich currents shift. Conversely, sharks often migrate into unprotected waters where they face high risks of incidental bycatch from industrial longline fisheries.

Currently, there is a lack of accessible, real-time tools that can predict where these animals are likely to be. Researchers rely on historical tagging data, which tells us where a shark *was*, not where it *is* or *will be*. Fishery managers lack "Operational Oceanography" tools to implement **Dynamic Ocean Management (DOM)**—a strategy where protection zones move in sync with the animals.

1.3 Objective of the Project

The primary objective of **SharkTrack AI** is to bridge this technological gap. Our goal is to develop a high-accuracy, globally applicable shark foraging hotspot prediction system that operates in near real-time.

Specific Objectives:

1. **To Develop a Predictive Model:** Train a Machine Learning model (XGBoost) using open-source satellite data (NASA MODIS & NOAA) to predict the "Foraging Suitability Index" of any ocean coordinate.

2. **To Simulate Telemetry:** Create a robust IoT simulation layer that mimics the behavior of satellite-tagged sharks, allowing for the testing of multi-agent tracking systems.
3. **To Build a Real-Time Dashboard:** Architect a full-stack web application that visualizes both the live position of tracked animals and the predicted suitability of their surrounding habitat.
4. **To Enable Dynamic Management:** Provide a tool that allows conservationists to identify "High Risk" zones where shark foraging overlaps with human activity.

1.4 Project Scope and Limitations

Geographical Scope:

The system is designed for Global Wall-to-Wall Prediction. The underlying ML model creates a 4km-resolution grid applicable to any open-ocean environment. However, specific validation case studies focus on the Indian Ocean and South African waters (e.g., Mossel Bay, False Bay), known for their high white shark density and complex oceanography.

Technical Scope:

Data Sources: The model relies strictly on **Open Data**. We utilize Chlorophyll-a from NASA's MODIS-Aqua sensor and Sea Surface Temperature (SST) from NOAA's Pathfinder dataset. We explicitly exclude proprietary bathymetry or can issue alerts to prevent bycatch.

1. **System Administrator:** Manages the backend health. They ensure the `shark_habitat_model.pkl` is loaded correctly and that the `/hotspots/real` endpoint is responsive.

Chapter 2 – System Description

2.1 Customer/User Profiles

- **Marine Biologists:** Expert users requiring precise data. They utilize the system to correlate "Transiting" vs. "Feeding" events generated by the telemetry simulator with the ML model's predictions.
- **Fishery Management Organizations (RFMOs):** Regulatory bodies needing "Operational Oceanography" tools. They rely on the dashboard's **Dark Mode UI**, designed for reduced eye strain in 24/7 control room environments.

- **NGOs:** Organizations using the visual heatmaps to advocate for dynamic marine zoning.
- **Developers:** The system is built for easy extension, allowing developers to plug in real IoT tag data streams replacing the simulator.

2.2 Assumptions and Dependencies

- **Satellite Data Availability:** The system depends on the availability of MODIS_Chlorophyll_2020_Mean.tif and NOAA_Pathfinder_SST_2020_Mean.tif.
- **Library Versions:** The production backend requires specific Python libraries: rasterio $\geq$ 1.3, joblib, and numpy $\geq$ 1.24 for thread-safe memory mapping.
- **Client Hardware:** The React frontend uses **Leaflet** assets loaded via CDN; a stable internet connection is assumed to prevent local build errors.
- **Concurrency:** The system assumes the host server supports multi-threading to handle independent shark simulation agents simultaneously.

2.3 Functional and Non-Functional Requirements

Functional Requirements

- **Ingestion Endpoint:** The system must provide an /events/ingest endpoint that accepts JSON payloads, validates them against Pydantic schemas (TagPayload), and writes to MongoDB asynchronously.
- **Real-Time Inference:** The backend must generate predictions for a 40x40 grid over the Indian Ocean region upon startup and serve them via /hotspots/real.
- **Live Polling:** The frontend must poll the API every **2 seconds** to update shark positions without refreshing the page.
- **Event Triggering:** The simulator must randomly generate "Transiting" or "Feeding" events based on defined probability thresholds.

Non-Functional Requirements

- **Latency:** The ML inference using rasterio.sample() must be optimized to run in real-time batches, minimizing dashboard lag.
- **Fault Tolerance:** The frontend must automatically fall back to simulated hotspots if the ML engine fails or if satellite rasters are corrupt.
- **Scalability:** The architecture must support **Multi-Agent Tracking**, creating independent

threads for different sharks.

- **Asynchronous Processing:** The backend must use `async/await` for all Database I/O to ensure the server does not freeze during high-frequency ingestion.

Chapter 3 – Design

3.1 System Design

The application follows a modern **Microservices-style Architecture** composed of four distinct modules:

1. **IoT Simulation Layer (`tag_simulator.py`):**
 - Mimics real-world conditions without live hardware.
 - **Behavioral Logic:** Implements "Random Walk" for geospatial movement and "Sine Wave" functions for depth profiles.
 - **Data Payload:** Generates JSON containing GPS coordinates, depth, acceleration, and battery status.
2. **Backend API (`main.py`, `events.py`):**
 - Built on **FastAPI** for speed and automatic documentation.
 - Acts as the controller, managing data ingestion and validation (ensuring ISO-8601 timestamps and float coordinates).
3. **ML Prediction Engine (`ml_predictor.py`):**
 - The scientific core. Loads the `.pkl` model and GeoTIFFs once at startup.
 - Uses **Chunked Rasterio + Numpy Memmap** to handle large satellite files efficiently.
 - Generates the geospatial grid and runs inference to populate the "Suitability Index."
4. **Frontend Dashboard (`App.jsx`):**
 - A **React** Single Page Application (SPA).
 - Uses `react-leaflet` with custom icons to render shark movements.
 - Manages state via React hooks (`useState`, `useEffect`) to toggle map layers and handle live feeds.

3.1.1 E–R Diagram

The Entity-Relationship diagram focuses on the NoSQL structure in MongoDB:

- **Tag_Events Collection:** Stores high-volume telemetry. Fields: tag_id (SHK001), timestamp, lat, lon, depth, status (feeding/transiting).
- **Grid_Predictions Collection:** Stores the calculated hotspots. Fields: grid_id, lat, lon, suitability_score, generated_at.
- **Satellite_Metadata:** Stores file paths and bounds for the loaded GeoTIFFs.

3.1.2 Data Flow Diagrams (DFDs)

Level 1 Data Flow:

1. **Simulator** generates coordinates -> Sends POST to /events/ingest.
2. **API** validates payload -> Writes to **MongoDB**.
3. **Frontend** Polls /events/latest -> API queries **MongoDB** -> Returns JSON.
4. **Frontend** renders Shark Icon at new coordinates.
5. **Frontend** requests /hotspots/real -> **ML Engine** samples GeoTIFFs -> Returns Grid -> **Frontend** renders Heatmap.

3.2 Database Design

MongoDB is chosen for its ability to ingest high-velocity unstructured JSON data from tags.

- **Ingestion Optimization:** Indexing on tag_id and timestamp ensures fast retrieval for the live dashboard.
- **Spatial Data:** Coordinates are stored as standard GeoJSON points to allow for future geospatial queries (e.g., "Find all sharks within this polygon").

3.3 Machine Learning Pipeline

This section details the technical implementation of the "Sharks from Space" model.

3.3.1 Dataset Summary (2020 Global Training Data) The model was trained on a curated dataset combining biological presence with environmental drivers.

Layer	Source	Resolution	Type	Exported File
Chlorophyll-a	NASA	4 km	Annual Mean	MODIS_Chlor

	MODIS-Aqua			ophyll_2020_Mean.tif
SST	NOAA Pathfinder v5.3	4 km	Annual Mean	NOAA_Pathfinder_SST_2020_Mean.tif
Shark Presence	International Shark Attack File (ISAF)	Point	312 records	(Unprovoked only)
Pseudo-absences	Random Ocean Points	Point	10,000	Weighted MaxEnt-style

3.3.2 Feature Engineering & Selection We tested 7 candidate features (including distance-to-coast, bathymetry, and gradients). Through permutation importance analysis, the final model was streamlined to use **only two features**:

1. **Chlorophyll-a**: The strongest proxy for prey availability (phytoplankton abundance).
2. **Sea Surface Temperature (SST)**: The strongest signal for thermal preference.

3.3.3 Model Comparison We evaluated multiple algorithms using 5-fold spatial block cross-validation.

Algorithm	ROC-AUC	Accuracy	RMSE	Training Time
XGBoost Regressor	0.914	91.8 %	0.112	22 s
Random Forest	0.891	89.3 %	0.129	14 s
Gradient Boosting	0.907	90.7 %	0.118	18 s

Neural Network (MLP)	0.883	88.6 %	0.141	41 s
----------------------------	-------	--------	-------	------

Winner: XGBoost Regressor was selected for the best balance of accuracy and interpretability.

3.3.4 Final Model Hyperparameters

The final model (shark_habitat_model.pkl) uses the following configuration:

- n_estimators: 800
- max_depth: 7
- learning_rate: 0.05
- subsample: 0.9
- colsample_bytree: 0.9
- objective: 'reg:squarederror'

3.3.5 Independent Validation

The model was validated against an independent dataset of 18 satellite-tagged white sharks in South Africa (2021–2023).

- **Result:** 87% of independent tag positions fell within the top 20% predicted suitability zones.
- **Hotspots Identification:** The model correctly identified known upwelling zones in **Mossel Bay, False Bay, and Gansbaai**.

Chapter 4 – Scheduling and Estimates

4.1 Project Scheduling

1. **Data Acquisition (Weeks 1-3):** Exporting MODIS and NOAA data from Google Earth Engine.
2. **Model Training (Weeks 4-7):** Training XGBoost, tuning hyperparameters, and exporting shark_habitat_model.pkl.
3. **Backend Dev (Weeks 8-10):** Building main.py and events.py, integrating rasterio for GeoTIFF handling.

4. **Frontend Dev (Weeks 11-13):** Developing the React Dashboard and Leaflet integration.
5. **Integration & Testing (Weeks 14-16):** connecting the tag_simulator.py to the live API and validating the fallback mechanisms.

4.2 Resource Allocation and Task Estimation

- **ML Team (Abdul, Rajdeep):** 100% completed. Tasks: Feature engineering, Model comparison, Production integration (backend/real_ml_predictor.py).
- **Full Stack Team (Abhishek):** 100% completed. Tasks: API Architecture, Database setup, React UI, Dockerization.
- **Data Science Team (Mansi, Sania):** 100% completed. Tasks: Patent drafting ("Adaptive Oceanic Telemetry"), Documentation, Validation studies.

Conclusion

The **SharkTrack AI** project successfully demonstrates how modern web technologies can be combined with Data Science to create actionable tools for marine conservation. By simulating realistic telemetry and integrating environmental prediction models, the system provides a robust prototype for future deployment with real satellite tags.

The ML team has completed its mission: the model is trained, validated, globalized, and live in production. The final XGBoost model achieved an **ROC-AUC of 0.914**, successfully predicting global hotspots using only open-source data.

Future Scope:

1. **Historical Playback:** Adding a "scrubber" to replay shark movements over the past week.
2. **Alert System:** Sending SMS/Email alerts when a shark enters a high-risk zone.
3. **3D Visualization:** Rendering the depth profile in a 3D chart to visualize diving behavior alongside the 2D map.
4. **Live IoT Integration:** Replacing the simulator with real satellite tag feeds via MQTT or Iridium.